

CS620T: Compilers - Principles and Implementation

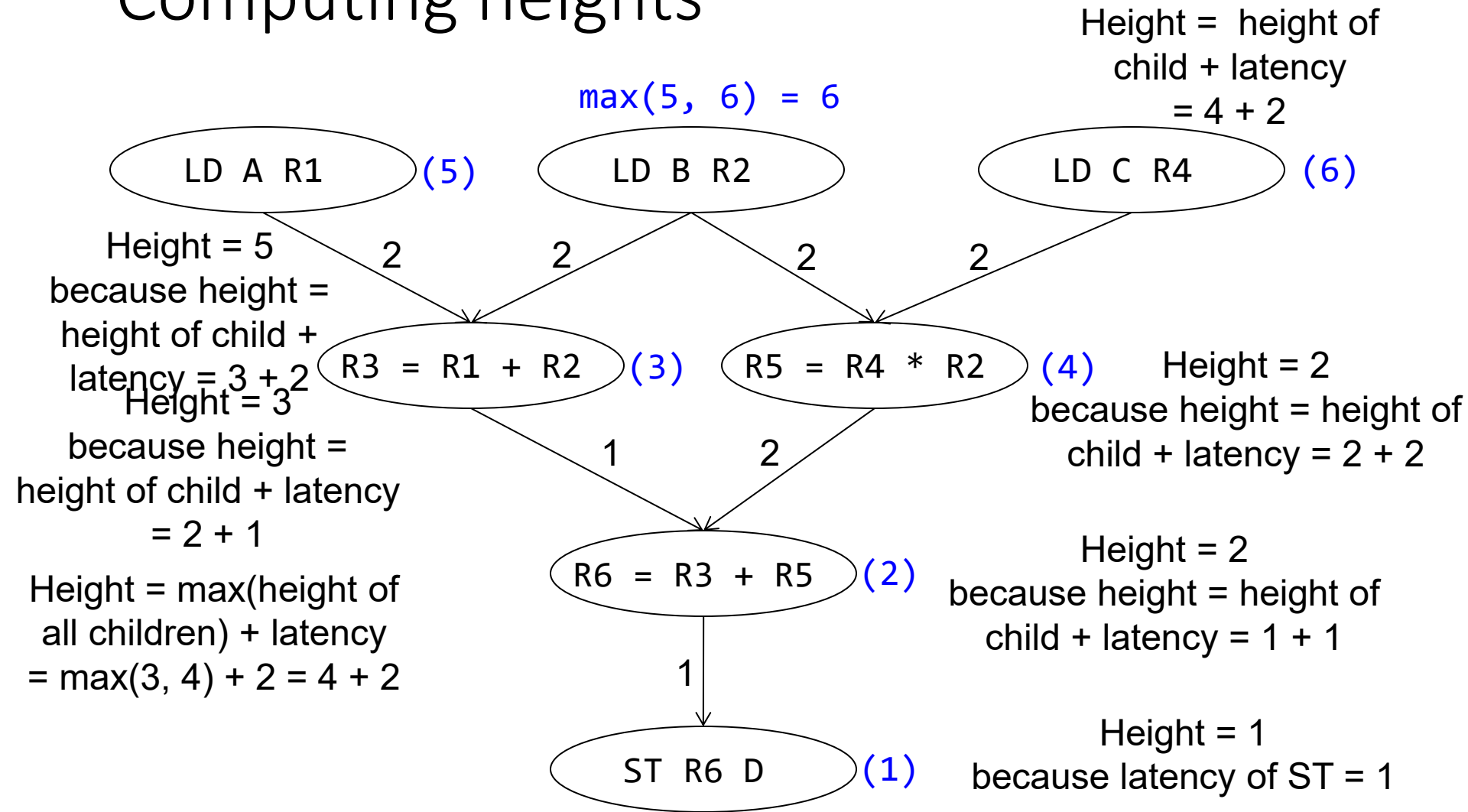
Autumn 2026

Week 9: List Scheduling (contd.), Basic
Blocks, CFG,

Height-based scheduling

- Important to prioritize instructions
 - Instructions that have a lot of downstream instructions dependent on them should be scheduled earlier
- Instruction scheduling NP-hard in general, but **height-based scheduling** is effective
- Instruction height = latency from instruction to farthest-away leaf
 - Leaf node height = instruction latency
 - Interior node height = $\max(\text{heights of children} + \text{instruction latency})$
- Schedule instructions with highest height first

Computing heights



Height-based list scheduling

1. LD A, R1
2. LD B, R2
3. $R3 = R1 + R2$
4. LD C, R4
5. $R5 = R4 * R2$
6. $R6 = R3 + R5$
7. ST R6, D

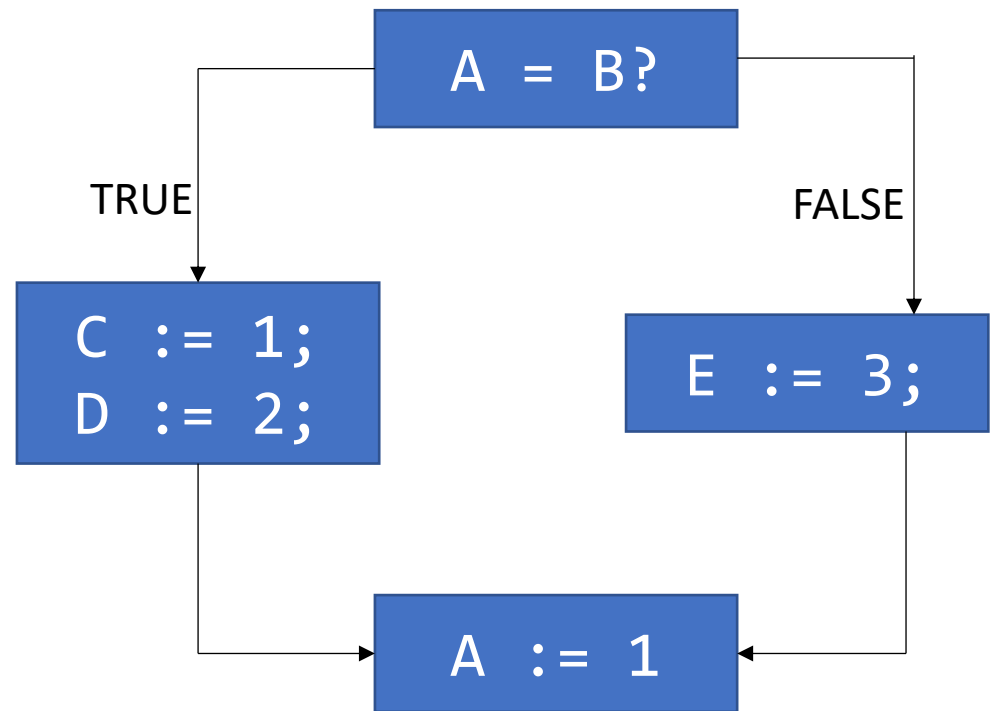
Cycle	ALU0	ALU1	LD/ST
0			2
1			2
2			4
3			4
4	5		1
5			1
6	3		
7	6		
8	7		
9			
10			

Basic Blocks and Flow Graphs

- Basic Block
 - Maximal sequence of consecutive instructions with the following properties:
 - The first instruction of the basic block is the *only entry point*
 - The last instruction of the basic block is either the halt instruction or the *only exit point*
- Flow Graph
 - Nodes are the basic blocks
 - Directed edge indicates which block follows which block

Basic Blocks and Flow Graphs - Example

```
if A = B then  
    C := 1;  
    D := 2;  
else  
    E := 3  
fi  
A := 1;
```



A data flow graph

Flow Graphs

- Capture how control transfers between basic blocks due to:
 - Conditional constructs
 - Loops
- Are necessary when we want optimize considering larger parts of the program
 - Multiple procedures
 - Whole program

Flow Graphs - Representation

- We need to label and track statements that are jump targets
 - **Explicit targets** – targets mentioned in jump statement
 - **Implicit targets** – targets that follow conditional jump statement
 - Statement that is executed if the branch is not taken
- Implementation
 - Linked lists for Basic Blocks
 - Graph data structures for flow graphs

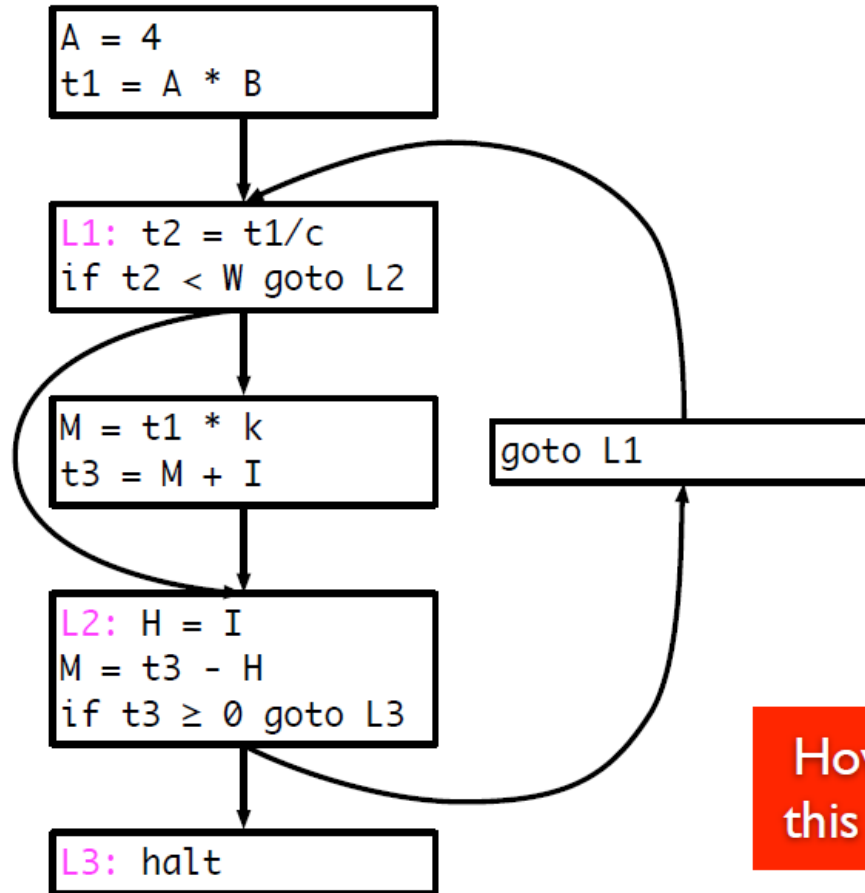
Running example

```
A = 4
t1 = A * B
repeat {
  t2 = t1/C
  if (t2 ≥ W) {
    M = t1 * k
    t3 = M + I
  }
  H = I
  M = t3 - H
} until (T3 ≥ 0)
```

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

CFG for running example



How do we build
this automatically?

Constructing a CFG

- To construct a CFG where each node is a basic block
 - Identify *leaders*: first statement of a basic block
 - In program order, construct a block by appending subsequent statements up to, but not including, the next leader
- Identifying leaders
 - First statement in the program
 - Explicit target of any conditional or unconditional branch
 - Implicit target of any branch

Partitioning algorithm

- Input: set of statements, $stat(i)$ = i^{th} statement in input
- Output: set of *leaders*, set of basic blocks where $block(x)$ is the set of statements in the block with leader x
- Algorithm

```
leaders = {1}           //Leaders always includes first statement
for i = 1 to |n|       //|n| = number of statements
    if  $stat(i)$  is a branch, then
        leaders = leaders  $\cup$  all potential targets
end for
worklist = leaders
while worklist not empty do
    x = remove earliest statement in worklist
    block(x) = {x}
    for (i = x + 1; i  $\leq$  |n| and i  $\notin$  leaders; i++)
        block(x) = block(x)  $\cup$  {i}
    end for
end while
```

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = ?

Basic blocks = ?

```
leaders = {1}           //Leaders always includes first statement
for i = 1 to |n|        //|n| = number of statements
    if stat(i) is a branch, then
        leaders = leaders ∪ all potential targets
end for
worklist = leaders
```

Running example

1	A = 4
---	-------

```
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1}

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1}

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3}

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3}

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5}

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5}

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7}

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7}

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7}

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10}

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11}
Basic blocks = ?

worklist = leaders

while *worklist* not empty **do**

x = remove earliest statement in *worklist*

block(x) = {*x*}

for (*i* = *x* + 1; *i* ≤ |*n*| and *i* ∉ *leaders*; *i*++)

block(x) = *block(x)* ∪ {*i*}

end for

end while

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11}

Basic blocks =

Block(1) = ?

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11}

Basic blocks =

Block(1) = ?

Start from statement 2 and add
till either the end or a leader is
reached

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11} Block(1) = {1, 2}

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11} Block(3) = ?

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11} Block(3) = {3, 4}
Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11} Block(5) = ?

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11} Block(5) = {5, 6}

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11} Block(7) = ?

Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11} Block(7) = {7, 8, 9}
Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11} Block(10) = ?
Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11} Block(10) = {10}
Basic blocks =

Running example

```
1      A = 4
2      t1 = A * B
3  L1:  t2 = t1 / C
4      if t2 < W goto L2
5      M = t1 * k
6      t3 = M + I
7  L2:  H = I
8      M = t3 - H
9      if t3 ≥ 0 goto L3
10     goto L1
11  L3:  halt
```

Leaders = {1, 3, 5, 7, 10, 11} Block(11) = {11}
Basic blocks =

Running example

1		A = 4
2		t1 = A * B
3	L1:	t2 = t1 / C
4		if t2 < W goto L2
5		M = t1 * k
6		t3 = M + I
7	L2:	H = I
8		M = t3 - H
9		if t3 ≥ 0 goto L3
10		goto L1
11	L3:	halt

Leaders = {1, 3, 5, 7, 10, 11}

Basic blocks = { {1, 2}, {3, 4}, {5, 6}, {7, 8, 9}, {10}, {11} }

Putting edges in CFG

- There is a directed edge from B_1 to B_2 if
 - There is a branch from the last statement of B_1 to the first statement (leader) of B_2
 - B_2 immediately follows B_1 in program order and B_1 does not end with an unconditional branch
- Input: *block*, a sequence of basic blocks
- Output: The CFG

```
for  $i = 1$  to  $|block|$      $\{\{1,2\}, \{3,4\}, \{5,6\}, \{7,8,9\}, \{10\}, \{11\}\}$   
     $x =$  last statement of  $block(i)$   
    if  $stat(x)$  is a branch, then  
        for each explicit target  $y$  of  $stat(x)$   
            create edge from block  $i$  to block  $y$   
        end for  
    if  $stat(x)$  is not unconditional then  
        create edge from block  $i$  to block  $i+1$   
    end for
```

Putting edges in CFG

- There is a directed edge from B_1 to B_2 if
 - There is a branch from the last statement of B_1 to the first statement (leader) of B_2
 - B_2 immediately follows B_1 in program order and B_1 does not end with an unconditional branch
- Input: *block*, a sequence of basic blocks
- Output: The CFG

```
for  $i = 1$  to  $|block|$      $\{\{1, 2\}, \{3, 4\}, \{5, 6\}, \{7, 8, 9\}, \{10\}, \{11\}\}$   
     $x = \text{last statement of } block(i)$     2:  $t1 = A * B$   
    if  $stat(x)$  is a branch, then  
        for each explicit target  $y$  of  $stat(x)$   
            create edge from block  $i$  to block  $y$   
        end for  
    if  $stat(x)$  is not unconditional then  
        create edge from block  $i$  to block  $i+1$   
end for
```

Edge from block 1 to block 2

Putting edges in CFG

- There is a directed edge from B_1 to B_2 if
 - There is a branch from the last statement of B_1 to the first statement (leader) of B_2
 - B_2 immediately follows B_1 in program order and B_1 does not end with an unconditional branch
- Input: *block*, a sequence of basic blocks
- Output: The CFG

```
for  $i = 1$  to  $|block|$      $\{\{1, 2\}, \{3, 4\}, \{5, 6\}, \{7, 8, 9\}, \{10\}, \{11\}\}$   
     $x = \text{last statement of } block(i)$     4: if  $t2 < W$  goto L2  
    if  $stat(x)$  is a branch, then  
        for each explicit target  $y$  of  $stat(x)$     Edge from block 2 to block 4  
            create edge from block  $i$  to block  $y$   
        end for  
    if  $stat(x)$  is not unconditional then  
        create edge from block  $i$  to block  $i+1$   
    end for
```

Putting edges in CFG

- There is a directed edge from B_1 to B_2 if
 - There is a branch from the last statement of B_1 to the first statement (leader) of B_2
 - B_2 immediately follows B_1 in program order and B_1 does not end with an unconditional branch
- Input: *block*, a sequence of basic blocks
- Output: The CFG

for $i = 1$ to $|block|$ $\{\{1, 2\}, \{3, 4\}, \{5, 6\}, \{7, 8, 9\}, \{10\}, \{11\}\}$

$x = \text{last statement of } block(i)$

if $stat(x)$ is a branch, **then**

for each explicit target y of $stat(x)$

 create edge from block i to block y

end for

if $stat(x)$ is not unconditional **then**

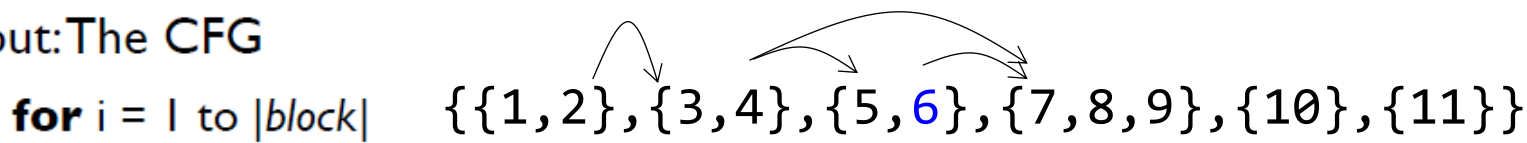
 create edge from block i to block $i+1$

end for

Edge from block 2 to block 3

Putting edges in CFG

- There is a directed edge from B_1 to B_2 if
 - There is a branch from the last statement of B_1 to the first statement (leader) of B_2
 - B_2 immediately follows B_1 in program order and B_1 does not end with an unconditional branch
- Input: *block*, a sequence of basic blocks
- Output: The CFG



```

for  $i = 1$  to  $|block|$ 
     $x = \text{last statement of } block(i)$ 
     $6: t3 = M + I$ 
    if  $stat(x)$  is a branch, then
        for each explicit target  $y$  of  $stat(x)$ 
            create edge from block  $i$  to block  $y$ 
        end for
    if  $stat(x)$  is not unconditional then
        create edge from block  $i$  to block  $i+1$ 
    end for
    
```

Edge from block 3 to block 4

Putting edges in CFG

- There is a directed edge from B_1 to B_2 if
 - There is a branch from the last statement of B_1 to the first statement (leader) of B_2
 - B_2 immediately follows B_1 in program order and B_1 does not end with an unconditional branch
- Input: *block*, a sequence of basic blocks
- Output: The CFG

for $i = 1$ to $|block|$ $\{\{1, 2\}, \{3, 4\}, \{5, 6\}, \{7, 8, 9\}, \{10\}, \{11\}\}$

$x = \text{last statement of } block(i)$

if $stat(x)$ is a branch, **then**

for each explicit target y of $stat(x)$

 create edge from block i to block y

end for

if $stat(x)$ is not unconditional **then**

 create edge from block i to block $i+1$

end for

Edge from block 4 to block 6

Putting edges in CFG

- There is a directed edge from B_1 to B_2 if
 - There is a branch from the last statement of B_1 to the first statement (leader) of B_2
 - B_2 immediately follows B_1 in program order and B_1 does not end with an unconditional branch
- Input: *block*, a sequence of basic blocks
- Output: The CFG

for $i = 1$ to $|block|$ $\{\{1, 2\}, \{3, 4\}, \{5, 6\}, \{7, 8, 9\}, \{10\}, \{11\}\}$
 $x = \text{last statement of } block(i)$
 if $stat(x)$ is a branch, **then**
 for each explicit target y of $stat(x)$
 create edge from block i to block y
 end for
 if $stat(x)$ is not unconditional **then**
 create edge from block i to block $i+1$
 end for

Edge from block 4 to block 5

Putting edges in CFG

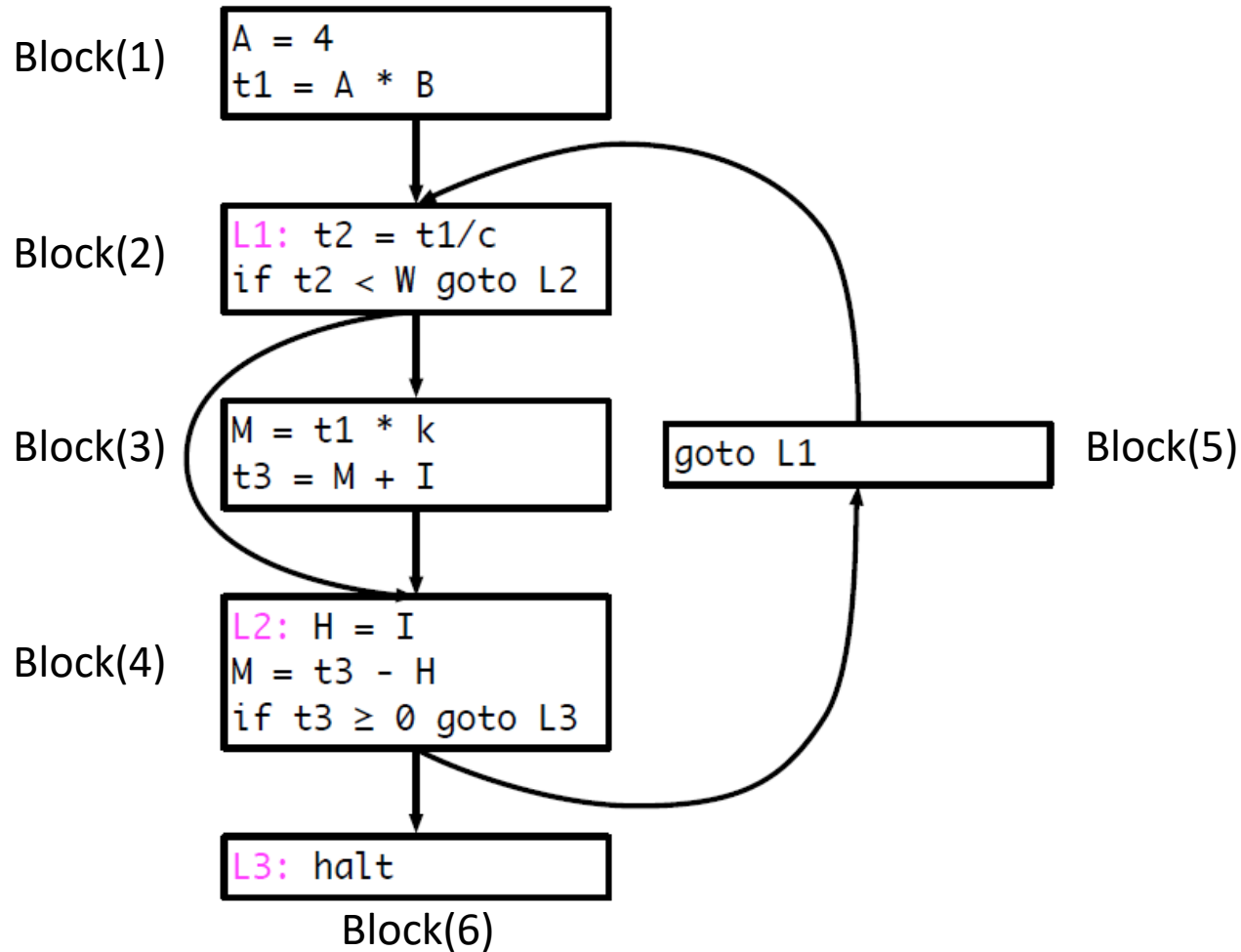
- There is a directed edge from B_1 to B_2 if
 - There is a branch from the last statement of B_1 to the first statement (leader) of B_2
 - B_2 immediately follows B_1 in program order and B_1 does not end with an unconditional branch
- Input: *block*, a sequence of basic blocks
- Output: The CFG

```

for  $i = 1$  to  $|block|$      $\{\{1, 2\}, \{3, 4\}, \{5, 6\}, \{7, 8, 9\}, \{10\}, \{11\}\}$ 
     $x = \text{last statement of } block(i)$ 
    if  $stat(x)$  is a branch, then
        for each explicit target  $y$  of  $stat(x)$ 
            create edge from block  $i$  to block  $y$ 
        end for
    if  $stat(x)$  is not unconditional then
        create edge from block  $i$  to block  $i+1$ 
    end for
    
```

Edge from block 5 to block 2

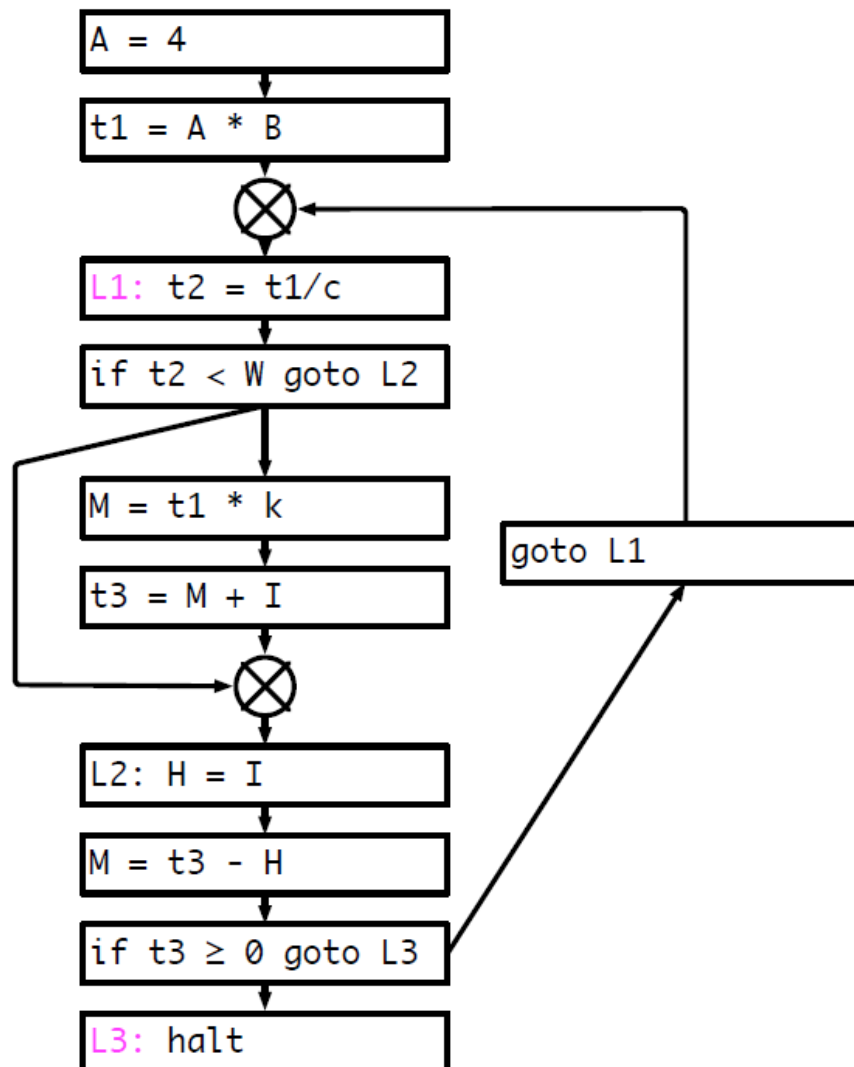
Result



Discussion

- Some times we will also consider the *statement-level* CFG, where each node is a statement rather than a basic block
- Either kind of graph is referred to as a CFG
- In statement-level CFG, we often use a node to explicitly represent *merging* of control
- Control merges when two different CFG nodes point to the same node
- Note: if input language is *structured*, front-end can generate basic block directly
- “GOTO considered harmful”

Statement level CFG



Control Flow Graphs - Use

- Why do we need CFGs? - Global Optimization
 - Optimizing compilers do global optimization (i.e. optimize beyond basic blocks)
 - Differentiating aspect of normal and optimizing compilers

Next: Techniques that help in Global Optimization

Content

- SSA
- CFGs
- Dominator trees and Dominance frontiers
- Application of concepts: loop invariant code motion

Recall: Single assignment form (motivation)

Single assignment form: a variable is assigned only once i.e. appears only once in LHS.

Aids CSE: $x = z + y$
...
 $x = z + y$

Neither z nor y can appear on the LHS here in single assignment form.

So, can be sure that this $z + y$ is the same expression as earlier. In the original code, if z or y were assigned to in between the two expressions, then we would have used different names, say, $z1 = ..$; $y1 = ..$; then the last expression would have to be rewritten as $x = z1 + y1$.

$x = z + y$
 $a = x$
 $x = 2 * x$

replace x with b



$b = z + y$
 $a = b$
 $x = 2 * b$

Aids copy propagation: can replace all the uses of a variable downstream

Aids dead code elimination: if the variable is never used later, can safely remove the statement where the variable is defined/assigned to.

SSA

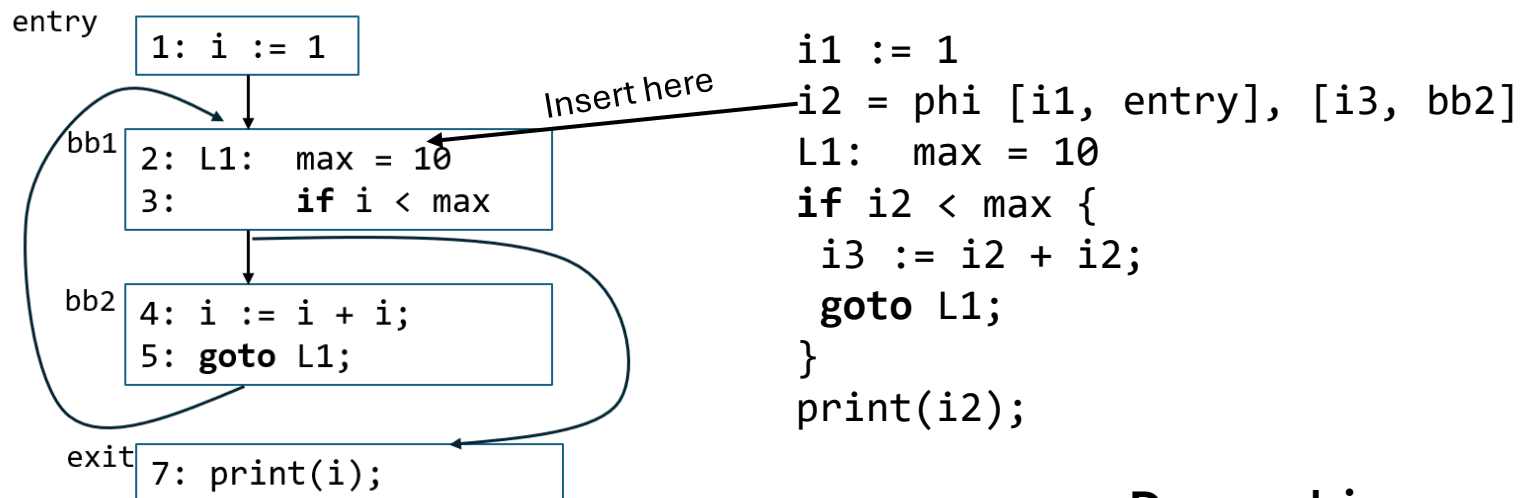
- When there is control flow, tricky to get SSA form.

```
1:      i1 = 1
2: L1:  max = 10
3:      if i1 < max {
4:          i2 = i1 + i1
5:      goto L1;
6:      }
7:      print(i);
```

i1 or i2 ?

Phi nodes – special instructions that help deal with control flow. E.g.,
 $i3 = \text{phi } [i1, \text{bb1}], [i2, \text{bb4}]$

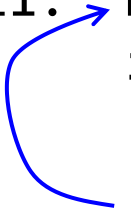
- Exercise:** draw the corresponding CFG. Insert phi node



SSA

- Converting from unrestricted form to SSA form

```
1:      i1 = 1
2: L1:  max = 10
3:      if i1 < max {
4:          i2 = i1 + i1
5:      goto L1;
6:      }
7:      print(i);
```



i1 or i2 ?

Phi nodes – special instructions that help deal with control flow. E.g.,
i3 = phi [i1, bb1], [i2, bb4]

- Where should we insert phi nodes?

Dominance frontiers

- What do we need to do after inserting phi nodes?

Rename variables so that every assignment gets a unique name

Dominators

- Describe **relationship** between basic blocks
 - A block dominates other if it is guaranteed to execute before the other
 - **Formally:**
 - if all paths from entry of CFG to node B pass through node A then we say that A dominates B
 - The relationship is reflexive i.e. node B dominates itself

Dominator Tree

- A data structure for tracking dominator relation
- A node in the tree dominates all the nodes of the subtree, for which the node is the root
- Terminology:
 - **Strict domination**: A strictly dominates B if it dominates B and $A \neq B$
 - **Immediate domination**: A dominates B and does not strictly dominate any other node that strictly dominates B (e.g. A is B's parent in the dominator tree)
 - **Domination frontier (DF)**: B is in the DF of A if
 - A does not dominate B but
 - dominates a predecessor of B
 - **Post domination**: A post-dominates B if on *all* paths from B to the exit node, A appears.

Finding Dominators

Output: dom = {map of vertex -> vertex set}

dom={}

while dom is still changing {

for vertex **in** CFG {

$dom[vertex] = vertex \cup$

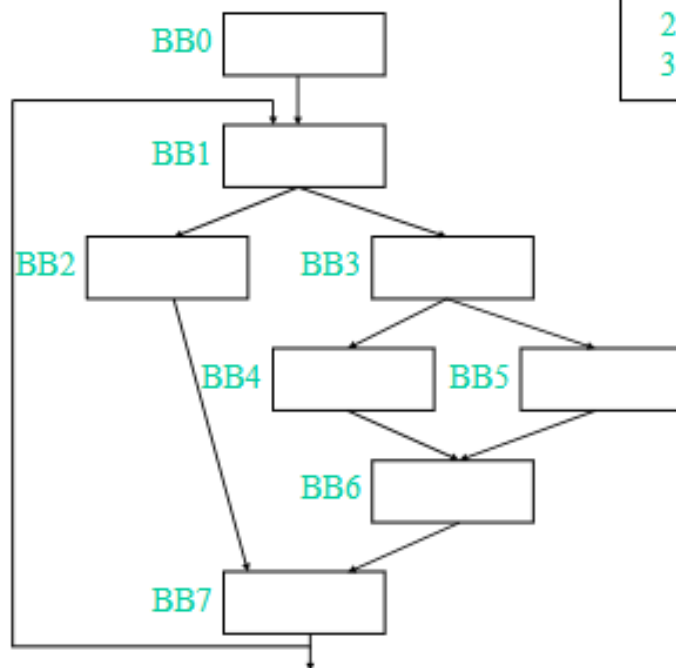
$\cap \{ dom[p], \text{where } p \in predecessor(vertex) \}$

 }

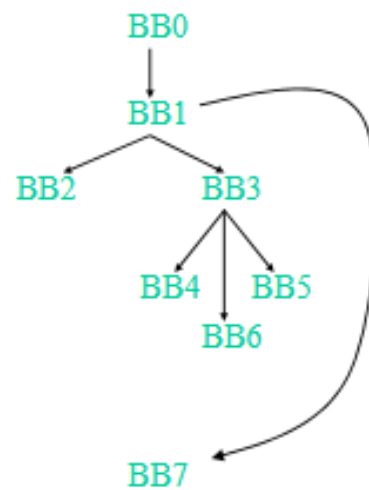
}

Dominator Tree

First BB is the root node, each node dominates all of its descendants



BB	DOM	BB	DOM
0	0	4	0,1,3,4
1	0,1	5	0,1,3,5
2	0,1,2	6	0,1,3,6
3	0,1,3	7	0,1,7



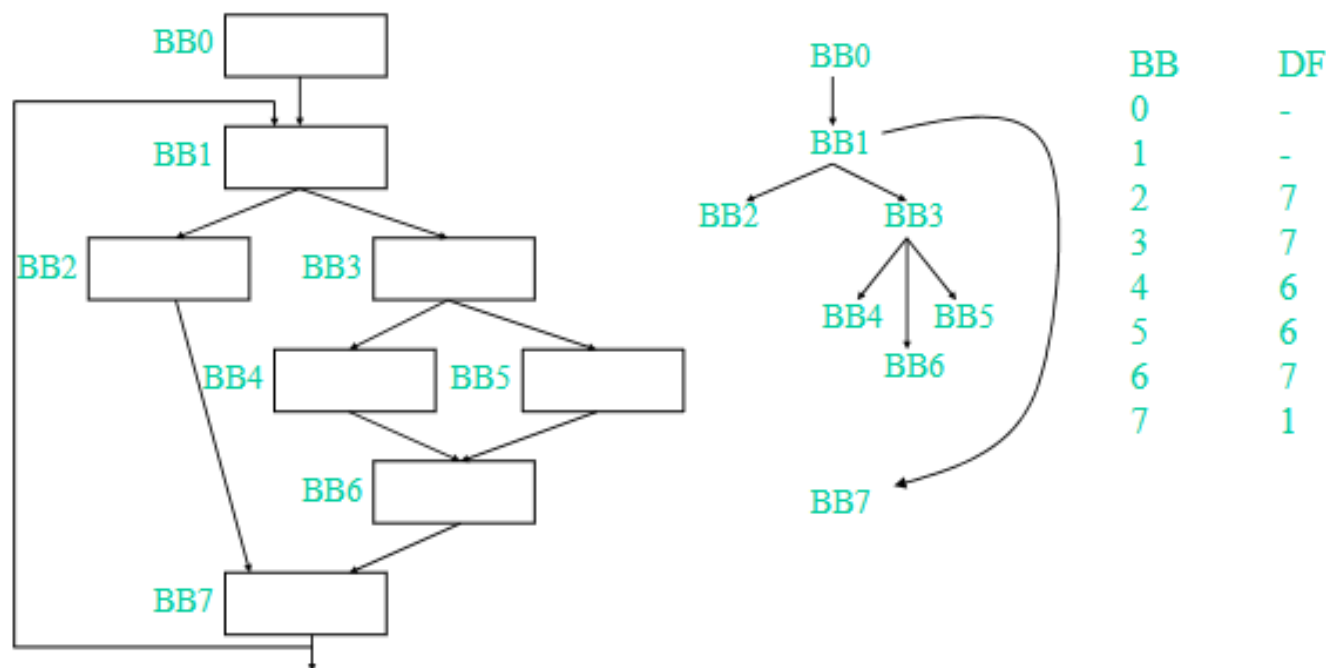
Dom tree

Finding Dominance Frontiers

Recall:

- Dominance frontier of a node X is the set of nodes Y such that
 - X dominates a predecessor of Y
 - X does not strictly dominate Y

Computing Dominance Frontiers



For each join point X in the CFG
For each predecessor of X in the CFG
Run up to the $IDOM(X)$ in the dominator tree,
adding X to $DF(N)$ for each N between X and $IDOM(X)$

Finding Dominators - Complexity

Output: dom = {map of vertex -> vertex set}

```
dom={}
```

```
while dom is still changing {  
    for vertex in CFG {  
        dom[vertex] = vertex  $\cup$   
         $\cap \{ dom[p], \text{where } p \in predecessor(vertex) \}$   
    }  
}
```

- Complexity? $O(n^2)$ n is number of vertices.
- Optimization (Linear time):
 - postorder traversal of CFG,
 - reverse the postordered list
 - process vertices in reverse order

Applicable to programming languages with *reducible* CFGs

Reducible CFG

- *If every back edge has a natural loop, then the CFG is reducible.*
- *A programming language with control flow constructs and without goto yields CFGs that are reducible*

Application of Concepts – Loop Optimization

- Loops are the most frequent targets of global optimization (because they are often the “hot-spots” during program execution)

how do we identify loops in CFGs?

Natural loops

- There is a cycle in the CFG (necessary but not sufficient)
- There is only one vertex with in-edges from outside the set. That one vertex is called the loop-entry/loop-header block. (also the set needs to be strongly connected i.e. there is a path from every vertex to every other vertex in the set).
- For a *backedge* $A \rightarrow B$, the *smallest* set of vertices L including A and B such that $\forall v \in L, predecessors(v) \subseteq L$ or $v = B$
 - **Backedge** – an edge from A to B , where B dominates A

- How to find backedges?

Do a breadth-first traversal of CFG. Edges that you encounter when discovering new vertices are forward edges. Everything else are backedges.

Identify Loops in CFGs (Simple)

- Loops – **how do we identify loops in CFGs?**

For a set of nodes, L , that belong to loop:

- 1) There is a *loop entry node* with the property that no other node in L has a predecessor outside L . That is, every path from entry of the entire flow graph (*graph entry node*) to any node in L goes through the loop entry node.
- 2) *Every node in L* has a non-empty path, completely within L , to the entry of L .